

الفحص الشامل لنظام عقلان لطب الأسنان

تحليل كامل للكيانات وقاعدة البيانات والخدمات والواجهة الأمامية
والترابط بين الوحدات مع خطة إصلاح مفصلة

المشروع: Aqlan Dental Pro

التاريخ: 20 مايو 2026

الإصدار: 1.0

المحلل: Z.ai Audit System

AUDIT

فهرس المحتويات

1. الملخص التنفيذي

2. إحصائيات المشاكل المكتشفة

3. فحص كيانات قاعدة البيانات والعلاقات

3.1 مشاكل حرجة في لقطة النموذج

3.2 علاقات مفقودة ومكسورة

3.3 مشاكل سلامة البيانات

3.4 الترابط: الفاتورة والدفع

3.5 الترابط: الحجز والمريض

3.6 الترابط: الموعد والطابور

3.7 نمط الحذف الناعم

4. فحص المتحكمات والخدمات

4.1 مشاكل حرجة في التدفق

4.2 مشاكل عالية الأولوية

4.3 مشاكل متوسطة الأولوية

5. فحص الواجهة الأمامية

5.1 مشاكل حرجة في الأنواع

5.2 مشاكل عالية الأولوية

5.3 مشاكل متوسطة الأولوية

6. تحليل تدفق الترابط بين الوحدات

7. خطة الإصلاح حسب الأولوية

1. الملخص التنفيذي

تم إجراء فحص شامل لنظام عقلان لطب الأسنان بعد دمج عدة مراحل تطويرية متتالية (المراحل 5 و 9 و 10 بالإضافة الحجز والمواعيد والطابور) بشكل سريع ومباشر إلى الفرع الرئيسي دون مراجعة كافية. كشف الفحص عن **67 مشكلة** ومنخفضة الخطورة. أخطر المشاكل المكتشفة تشمل: لقطة نموذج قاعدة البيانات تالفة بتعريف مكرر، عدم وجود كيار والمدفوعات، تدفق تأكيد الحجز لا ينشئ مريضاً أو موعداً تلقائياً، وعدم مزامنة إلغاء الطابور مع حالة الموعد.

يحتوي النظام على 71 كيان، 44 متحكماً، أكثر من 20 خدمة، وأكثر من 80 صفحة ومكون في الواجهة الأمامية. النظام من عدم تناسق كبير في البنية التحتية والترابط بين الوحدات، مما يسبب سلوكاً غير متوقع للمستخدمين وفجوات في

2. إحصائيات المشاكل المكتشفة



0	3	8	6	3	كيانات قاعدة البيانات
2	3	10	6	3	المتحكمات والخدمات
5	2	6	6	1	الواجهة الأمامية
0	2	4	4	-	الترابط بين الوحدات
7	10	28	22	7	الإجمالي

3. فحص كيانات قاعدة البيانات والعلاقات

3.1 مشاكل حرجة في لقطة النموذج (Model Snapshot)

DB-1# حرج لقطه النموذج تالفة - تعريف مكرر ل BookingRequest

ملف `AppDbContextModelSnapshot.cs` يحتوي على تعريفين لـ `BookingRequest`: النسخة الجديدة (مع `DeletedAt` و `ConvertedToAppointmentId`) والنسخة القديمة (بدون حقول الحذف الناعم ومع `Status` كنص). لا تتجاوز الأولى وقت التشغيل.

التأثير: EF Core يعمل بمخطط قديم، الاستعلامات قد تفقد أعمدة موجودة فعلياً في قاعدة البيانات، عمليات الترحيل ستنتج الإصلاص: حذف الكتلة المكررة القديمة من اللقطه، ثم إعادة توليد الترحيل.

DB-2# حرج اللقطه خارج المزامنة بشكل كبير مع النموذج الفعلي

الخصائص التالية موجودة في الكيانات والإعدادات لكنها غير موجودة في اللقطه: `Appointment.RoomName`, `ArrivedAt`, `DeletedAt`, `InRoomAt`, `ServiceId`, `ClinicRoomId` - `Visit.ServiceId`, `CheckoutStatus` - `Payment.InvoiceId` - `BookingRequest.DoctorId`

التأثير: تشغيل ترحيل جديد سيحاول إضافة أعمدة موجودة مسبقاً، مما يسبب فشل وقت التشغيل.
الإصلاص: حذف اللقطه وإعادة توليدها مع ترحيل توفيق.

DB-3# حرج عدم وجود كيان InvoicePayment للربط متعدد إلى متعدد

التصميم الحالي يستخدم `Payment.InvoiceId` كعلاقة مباشرة (واحد إلى متعدد)، مما يعني أن دفعة واحدة لا يمكن توليد فواتير. هذا يقيد بشكل كبير المرونة المالية للنظام.

التأثير: مريض يدفع 1000 ريال مقابل 3 فواتير لا يمكن توزيع الدفعة بشكل صحيح دون إنشاء 3 سجلات دفع منفصلة، مما يولد الدفعات ويكسر التقارير المالية.

الإصلاص: إنشاء كيان `InvoicePayment` مع `InvoiceId` و `PaymentId` و `AppliedAmount` و `AppliedAt`.

3.2 علاقات مفقودة ومكسورة

DB-4# عالي BookingRequest ليس له مفتاح أجنبي ل Patient

الكيان يخزن `PatientName` و `PhoneNumber` كنصوص حرة بدون `PatientId`. عند تأكيد الحجز وتحويله إلى موعد بالسجل المرجعي للمريض.

التأثير: فجوة في سلامة البيانات، لا يمكن تتبع أي مريض ينتمي إليه الحجز.

الإصلاص: إضافة `Guid? PatientId` و `Patient? Navigation` إلى `BookingRequest` مع إنشاء `requestConfiguration.cs`

DB-5# عالي BookingRequest.ConvertedToAppointmentId بدون مفتاح أجنبي

الحقل موجود ك Guid? عاري بدون خاصية تنقل أو تكوين مفتاح أجنبي. حذف الموعد لن يفرغ هذا الحقل.

التأثير: مراجع يتيمة، خطر تلف البيانات.

الإصلاح: إضافة خاصية تنقل Appointment? وتكوين المفتاح الأجنبي.

DB-6# عالي 40 كيان بدون ملف تكوين Fluent API

الكيانات التالية ليس لها ملف تكوين: Contract, Receipt, Employee, Notification, Setting, PatientAccount, WhatsAppMessage, Document, OrthoCase, SurgeryCase, Prescription, RolePermission و 28 كيان يعني: لا فهارس للمفاتيح الأجنبية، لا قيود على طول النص، لا سلوك حذف متسلسل محدد.

التأثير: مفاتيح أجنبية بدون فهارس = استعلامات بطيئة، سلوك حذف غير متوقع.

الإصلاح: إنشاء ملفات تكوين للأولويات: Contract, Receipt, PatientAccount, OrthoCase.

DB-7# عالي حقول الحالة نصية سحرية (Magic Strings) بدلاً من تعدادات

الكيانات التالية تستخدم string Status بدلاً من Contract.Status="active", LabOrder.Status="sent", SurgeryCase.Status="scheduled", TreatmentStage.Status="pending", OrthoCase.Status="active", InternalReferral.Status="pending", WhatsAppMessage.Status="pending", InventoryAdjustment.AdjustmentType="manual" وغيرها.

التأثير: أخطاء إملائية ستؤدي لتلف البيانات بصمت، لا تحقق وقت الترجمة، قيم حالة غير متسقة.

الإصلاح: إنشاء تعدادات لكل نوع حالة واستخدام HasConversion في التكوين.

DB-8# متوسط Appointment و ClinicQueueItem - مصدران لنفس البيانات

Appointment يحتوي على RoomName و CalledAt و InRoomAt و ClinicQueueItem يحتوي أيضاً على نفس استدعاء مريض يجب تحديث كلا الكيانين.

التأثير: مصدران للحقيقة لنفس البيانات، إذا تباعدا يعرض النظام حالة خاطئة.

الإصلاح: إزالة حقول الطابور من Appointment واستخدام ClinicQueueItem كمصدر وحيد، أو إضافة آلية مزامنة.

DB-9# متوسط BookingRequest.PreferredDate عدم تطابق النوع مع الترحيل

الكيان يعرّف PreferredDate ك string? لكن الترحيل واللقطة القديمة تعرفه ك DateOnly?. هذا عدم تطابق يسبب وقت التشغيل.

التأثير: أخطاء في قراءة/كتابة طلبات الحجز.

الإصلاح: توحيد النوع إلى DateOnly? في الكيان والترحيل.

3.3 مشاكل سلامة البيانات

Payment.InvoiceId - سلوك SetNull يكسر مسار التدقيق عالي #DB-10

حذف فاتورة سيفرغ InvoiceId على جميع الدفعات المرتبطة عبر OnDelete(DeleteBehavior.SetNull)، مما يفقد كانت لأي فاتورة.

التأثير: مسار التدقيق المالي مكسور إذا حذفت فاتورة.

الإصلاح: تغيير إلى OnDelete(DeleteBehavior.Restrict) واستخدام الحذف الناعم بدلاً من ذلك.

ReceiptNumber بدون قيد تفرد متوسط #DB-11

Receipt.ReceiptNumber و Payment.ReceiptNumber ليس ليهما قيد تفرد. يمكن إنشاء أرقام إيصالات مكررة.

التأثير: إيصالات مكررة تسبب ارتباكاً في التقارير المالية.

الإصلاح: إضافة فهرس تفرد في التكوين.

Patient.Phone و WhatsApp قابلان للقيم الفارغة متوسط #DB-12

كلا الحقلين string? بدون تحقق إلزامي لمعلومات اتصال واحدة على الأقل. يمكن إنشاء مريض بدون أي وسيلة تواصل.

التأثير: متابعة المريض تصبح مستحيلة.

الإصلاح: إضافة تحقق على مستوى التطبيق يطلب رقم هاتف واحد على الأقل.

3.4 الترابط: الفاتورة والدفع

InvoiceStatus.Paid بدون آلية انتقال تلقائي عالي #DB-13

حالة Paid موجودة في التعداد لكن لا يوجد آلية تلقائية لانتقال الفاتورة من Issued إلى Paid عندما تساوي المدفوعات الفاتورة. التعليق في الكود يقول صراحة "هذه المرحلة لا تعين الحالة Paid تلقائياً".

التأثير: الفواتير تبقى في حالة Issued حتى بعد دفعها بالكامل، مما يجعل التقارير المالية غير دقيقة.

الإصلاح: إضافة خدمة مجال تفحص إجمالي المدفوعات مقابل إجمالي الفاتورة وتحديث الحالة تلقائياً عند تسجيل دفعات.

3.5 الترابط: الحجز والمريض

عالي #DB-14 تدفق تأكيد الحجز مكسور - لا إنشاء تلقائي للمريض

عند تأكيد طلب حجز: (1) لا يوجد PatientId للربط بمريض موجود، (2) ConvertedToAppointmentId بدون مفتاح آلية لإنشاء سجل مريض تلقائياً من بيانات الحجز، (4) BookingRequestStatus معرف داخل ملف الكيان بدلاً من التأثير: الموظفون يجب أن ينفذوا 2-3 خطوات يدوية لكل حجز مؤكد. إذا نسوا إنشاء المريض، يقع في الفجوة. الإصلاح: إضافة منطق إنشاء تلقائي للمريض في ConvertToAppointment، أو نقطة نهاية "تأكيد وإنشاء" بنقرة واحدة.

3.6 الترابط: الموعد والطابور

متوسط #DB-15 ClinicQueueStatus لا يحتوي على حالة NoShow

AppointmentStatus يحتوي على NoShow لكن ClinicQueueStatus لا يحتوي عليها. عندما يوضع موعد ك NoShow عنصر الطابور غير محددة وقد تبقى في حالة Waiting إلى أجل غير مسمى. التأثير: عناصر طابور يتيممة في حالة Waiting. الإصلاح: إضافة NoShow إلى ClinicQueueStatus أو إضافة منطق لإلغاء عنصر الطابور تلقائياً.

3.7 نمط الحذف الناعم

متوسط #DB-16 فلتر الحذف الناعم العالمي يطبق على جميع الكيانات بدون استثناء

الفلتر العالمي في AppDbContext يطبق IsActive على كل كيان يطبق ISoftDeletable. هذا يعني Permission تختفي من الاستعلامات عند IsActive=false، بما في ذلك إدارة المسؤول. التأثير: صلاحيات الدور المحذوفة ناعماً تختفي من كل مكان. الإصلاح: تجاوز الفلتر لكل كيان حسب الحاجة باستخدام IgnoreQueryFilters في الاستعلامات الإدارية.

4. فحص المتحكمات والخدمات

4.1 مشاكل حرجة في التدفق

API-1# حرج تأكيد الحجز لا ينشئ مريضاً أو موعداً

عند تغيير حالة طلب الحجز إلى "Confirmed"، النظام يحدث الحالة فقط ويرسل إشعار واتساب. لا ينشئ تلقائياً: (1) Patient، سجل (3) Appointment، عنصر ClinicQueue. يوجد نقطة نهاية منفصلة ConvertToAppointment تتطلب PatientId من المتصل.

التأثير: تدفق مكسور للمرضى الجدد. الموظف يجب أن ينفذ 2-3 خطوات يدوية. إذا نسي إنشاء المريض، الحجز يؤكد لكن لا المريض يسقط في الفجوة.

الإصلاح: في ConvertToAppointmentAsync، إضافة منطق إنشاء تلقائي للمريض إذا لم يوجد مطابق برقم الهاتف. أو إضافة "تأكيد وإنشاء" بنقرة واحدة.

API-2# حرج لا يوجد نقطة نهاية لإنشاء فاتورة

InvoicesController يحتوي على GET و PUT و PATCH لكن لا يوجد POST لإنشاء فاتورة جديدة. الطريقة الوحيدة لإنشاء عبر PatientJourneyController.CreateDraftInvoice المرتبط بسير عمل الزيارة.

التأثير: لا يمكن إنشاء فواتير مستقلة (لمنتجات، رسوم مختبر، خدمات غير مرتبطة بزيارة).

الإصلاح: إضافة POST /api/invoices مع CreateInvoiceRequest يحتوي على PatientId وبنود الفاتورة و VisitId اختياري.

API-3# حرج إلغاء الطابور لا يزامن حالة الموعد

عند إلغاء عنصر طابور في ClinicQueueController.Cancel، الكود لا يستدعي SyncAppointmentStatus (عكس EnterRoom و StartVisit و Complete التي تزامن جميعها). حالة الموعد تبقى بدون تغيير.

التأثير: الموعد يمكن أن يكون في حالة Waiting/Called/InRoom بينما عنصر الطابور Cancelled. المريض يظهر في قائمة كنشط رغم إزالته من الطابور.

الإصلاح: إضافة await SyncAppointmentStatus(item, AppointmentStatus.Cancelled) قبل eChangesAsync.Cancel.

4.2 مشاكل عالية الأولوية

عالي #4-API 8 متحكمات بدون طبقة خدمة - منطق أعمال مدمج

المتحكمات التالية تحقق AppDbContext مباشرة وتحتوي على كل المنطق مدمجاً: ClinicQueueController, InvoicesController, VisitsController, DoctorsController, SearchController, NotificationsController, PrescriptionsController, ReportsController, PatientJourneyController (و BookingRequestService و FinanceService).

التأثير: لا يمكن اختبار الوحدة لأي من هذه المتحكمات، قواعد الأعمال مبعثرة، تكرار الكود (مثل `ildPatientDisplayName`) أماكن).

الإصلاح: استخراج طبقات خدمة لكل متحكم: `ClinicQueueService, InvoiceService, VisitService, DoctorService, ReportService` وغيرها.

عالي #5-API تحديث حالة الموعد + إضافة الطابور بدون معاملة

عند تغيير حالة الموعد إلى Arrived/Waiting، يتم إضافة ClinicQueueItem بعد تحديث الحالة لكن بعملية `anges` منفصلة. لا توجد معاملة تلتف على العمليتين. إذا فشلت إضافة الطابور، الموعد يبقى في حالة Arrived بدون عنصر `د`. التأثير: موعد في حالة Arrived بدون إدخال طابور مقابل. المستخدم يرى نجاحاً لكن الطابور غير متسق.

الإصلاح: لف العمليتين في معاملة، أو نقل منطق إضافة الطابور إلى `AppointmentService.UpdateStatusAsync`.

عالي #6-API فحص تعارض الغرفة بعد حفظ الموعد

في `AppointmentsController.Create` و `Update`، الموعد يُحفظ أولاً عبر `service.CreateAsync()` ثم يُفحص تعارضه. إذا وُجد تعارض، الموعد سبق حفظه في قاعدة البيانات رغم إرجاع خطأ 409. التأثير: تعارض غرفة يُبلغ عنه لكن الموعد محفوظ فعلياً - بيانات غير متسقة.

الإصلاح: نقل فحص تعارض الغرفة إلى داخل `CreateAsync/UpdateAsync` قبل `SaveChanges`.

عالي #7-API PaymentDto لا يعين CreatedAt/UpdatedAt

دالة `MapPayment` في `FinanceService` لا تعين حقول `UpdatedAt` و `CreatedAt` في `PaymentDto` رغم وجوده ستكون دائماً فارغة.

التأثير: الواجهة لا يمكنها عرض تواريخ إنشاء/تعديل الدفعات. مسار التدقيق مكسور.

الإصلاح: إضافة (`CreatedAt = p.CreatedAt.ToString("yyyy-MM-dd")` و `UpdatedAt = p.UpdatedAt.ToString("yyyy-MM-dd")`) في `MapPayment`.

عالي #API-8 استرداد الدفع لا يعيد حالة الفاتورة

عند استرداد دفعة عبر `RefundPaymentAsync`، يتم إنشاء دفعة سلبية لكن لا يتم فحص ما إذا كان إجمالي المدفوع إجمالي الفاتورة لإعادة الحالة من `Paid` إلى `Issued`. فاتورة مستردة بالكامل تبقى في حالة `Paid`.

التأثير: تقارير مالية غير دقيقة، فواتير مدفوعة بالكامل رغم صافي دفع صفر.

الإصلاح: بعد إنشاء الاسترداد، فحص إجمالي المدفوع وإعادة الحالة إذا لزم.

4.3 مشاكل متوسطة الأولوية

متوسط #API-9 BatchUpdateStatus يتجاوز طبقة الخدمة

`AppointmentsController.BatchUpdateStatus` يقرأ المواعيد مباشرة من `db.Appointments`، يحقق الانتقالاً ويحدث الحالة - متجاوزاً `AppointmentService.UpdateStatusAsync`. المنطق الإضافي في الخدمة (مثل إضافة لا يُستدعى).

التأثير: تحديث حالة دفعي إلى `Arrived/Waiting` لن يضيف المرضى تلقائياً إلى الطابور.

الإصلاح: استدعاء `service.UpdateStatusAsync` لكل موعد في الحلقة، أو إضافة طريقة دفعية للخدمة.

متوسط #API-10 نقطة نهاية الطابور العام تكشف بيانات المرضى

`PublicController.GetQueue` و `ClinicQueueController.GetDisplay` هما `[AllowAnonymous]` ويكشفان أنواع المواعيد وأسماء الأطباء، أي شخص يمكنه تعداد مرضى اليوم بدون مصادقة.

التأثير: تسرب معلومات صحية شخصية بدون مصادقة.

الإصلاح: إزالة نقطة النهاية أو طلب مصادقة. إذا لشاشة انتظار، استخدام أرقام الطابور بدلاً من الأسماء.

متوسط #API-11 ReportsController - استعلامات كثيفة الذاكرة ونمط N+1

تقارير متعددة تحمل كل البيانات إلى الذاكرة للحساب بدلاً من استخدام `GROUP BY` على مستوى قاعدة البيانات. تحمل الأطباء منفصلاً يسبب نمط `N+1`. التقارير المتأثرة: التركيبة السكانية، العقود المتأخرة، الاحتفاظ بالمرضى، تحليل الأدو

التأثير: تدهور الأداء مع نمو البيانات. التقارير ستصبح بطيئة وجائعة للذاكرة.

الإصلاح: استخدام `GROUP BY` وتجميع على مستوى قاعدة البيانات، `Include()` لأسماء الأطباء.

متوسط #API-12 تحقق مفقود لعدة DTOs

FluentValidation: UpdateAppointmentStatusRequest, CreateVisitRequest, UpdateVisitRequest, IntakeRequest, HandoffRequest, CheckoutRequest, AddToQueueRequest, UpdateInvoiceRequest, CancelInvoiceRequest

التأثير: بيانات غير صالحة قد تصل لمنطق الأعمال.

الإصلاح: إضافة محقق FluentValidation لجميع DTOs المطلوبة.

5. فحص الواجهة الأمامية

5.1 مشاكل حرجية في الأنواع

حرج #FE-1 تأكيد الحجز قد يؤكد بدون ملف مريض

في booking-requests/page.tsx، عند تأكيد حجز، يتم استدعاء ensurePatientFile أولاً، إذا فشل، كتلة catch في api.patch لتأكيد الحجز (السطر 456). هذا ينشئ حجزاً مؤكداً بدون ملف مريض. الرسالة تقول "تم تأكيد الطلب لكن المريض" وهو مربك.

التأثير: حجز مؤكد بدون مريض مرتبط - الموظف لا يستطيع العثور على المريض.

الإصلاح: إذا فشل إنشاء المريض، لا تؤكد الحجز. بدلاً من ذلك، أظهر خطأ واترك الحجز في حالة المراجعة.

5.2 مشاكل عالية الأولوية

عالي #FE-2 useDashboardStats hook بأنواع قديمة لا تتطابق

الخطاف يعرف DashboardStats بخصائص مختلفة تماماً عن النوع الأساسي في types/dashboard.ts. صفحة لم تتجاوز الخطاف وتستدعي API مباشرة. الخطاف كود ميت بأنواع خاطئة.

التأثير: أي مكون يستخدم useDashboardStats() سيفشل في عرض البيانات الصحيحة.

الإصلاح: حذف الخطاف الميت أو تحديثه ليتطابق مع النوع الأساسي.

عالي #3-FE نوع QueueDisplay لا يتطابق مع استجابة API

نوع QueueDisplay في types/appointment.ts يستخدم patientId و appointmentId لكن استجابة API الفهم queueItemId. صفحة clinic-display تعرف واجهة DisplayData الخاصة بها. أي كود يستورد QueueDisplay حقول خاطئة.

التأثير: عدم تطابق أنواع يسبب أخطاء وقت التشغيل.

الإصلاح: تحديث نوع QueueDisplay ليتطابق مع استجابة API الفعلية.

عالي #4-FE PaginatedResponse مكرر بأشكال غير متوافقة

نوعان PaginatedResponse في types/api.ts و types/inventory.ts بحقول مختلفة: api.ts يستخدم totalCount و inventory.ts يستخدم total. استيراد من ملف خاطئ يسبب فشلاً.

التأثير: خطر سلامة الأنواع، سلوك غير متوقع حسب مصدر الاستيراد.

الإصلاح: توحيد في نوع PaginatedResponse واحد في types/api.ts.

عالي #5-FE صفحة المواعيد لا ترسل ServiceId و ClinicRoomId

نموذج إنشاء/تعديل الموعد في الواجهة الأمامية لا يرسل ServiceId و ClinicRoomId رغم أن الخلفية تتوقعهما. فحدهم الغرفة لا يعمل بدون ClinicRoomId.

التأثير: تعارضات غرف غير مكتشفة، مواعيد بدون خدمة محددة.

الإصلاح: إضافة حقول اختيار الخدمة والغرفة لنموذج الموعد.

عالي #6-FE أنواع Finance غير متطابقة مع DTOs الخلفية

الأنواع في types/finance.ts لا تتضمن InvoiceId في PaymentDto، ولا تتضمن InvoicePayments في InvoiceDto يعني الواجهة لا تعرض الربط بين الفواتير والمدفوعات.

التأثير: صفحة الفاتورة لا تعرض المدفوعات المرتبطة، لا تخصيص مدفوعات.

الإصلاح: تحديث أنواع Finance لتتطابق مع DTOs الخلفية.

5.3 مشاكل متوسطة الأولوية

متوسط #FE-7 معالجة أخطاء مفقودة في استدعاءات API

عدة صفحات لا تتعامل مع أخطاء API بشكل صحيح: صفحة المخزون لا تعالج أخطاء الترحيل، صفحة التقارير لا تعالج البيانات، صفحة الموردین لا تعالج أخطاء الإنشاء.

التأثير: تجميد الصفحة أو رسائل خطأ غير مفيدة عند فشل API.

الإصلاح: إضافة كتل try-catch مع رسائل toast لجميع استدعاءات API.

متوسط #FE-8 عدم تحديث البيانات بعد العمليات

بعد عمليات مثل تأكيد حجز أو إنشاء دفعة، بعض الصفحات لا تعيد تحميل البيانات. صفحة الحجز لا تعيد تحميل القائد صفحة الفواتير لا تتحدث بعد إصدار فاتورة.

التأثير: المستخدم يرى بيانات قديمة يجب تحديث الصفحة يدوياً.

الإصلاح: إعادة جلب البيانات بعد كل عملية تحويل ناجحة.

متوسط #FE-9 الشريط الجانبي لا يتضمن روابط لجميع الوحدات

الشريط الجانبي يفتقر لروابط لبعض الوحدات الجديدة: أوامر الشراء (Purchase Orders) غير موجودة كرابط مباشر، الطابور غير مرتبطة، صفحة رحلة المريض مخفية.

التأثير: المستخدمون لا يجدون الوحدات الجديدة بسهولة.

الإصلاح: تحديث الشريط الجانبي مع روابط لجميع الوحدات.

6. تحليل تدفق الترابط بين الوحدات

6.1 تدفق الحجز إلى المريض إلى الموعد إلى الطابور

التدفق الحالي في النظام يمر بالمراحل التالية، مع فجوات محددة:

1. نموذج الحجز العام

يُنشئ BookingRequest (Status: Pending) **يعمل** POST /api/public/booking-requests



2. تأكيد الحجز

يحدث الحالة لـ Confirmed ويرسل إشعار واتساب PATCH /api/booking-requests/{id}/status

لا ينشئ مريضاً - لا ينشئ موعداً



3. خطوة يدوية: إنشاء المريض

يجب على الموظف إنشاء المريض يدوياً عبر POST /api/patients
فجوة - يمكن نسيانها



4. تحويل الحجز لموعد

POST /api/booking-requests/{id}/convert-to-appointment يُنشئ Appointment ويضبط AppointmentId
يعمل - إذا لنفس اليوم: ClinicQueueItem يُنشأ تلقائياً



5. وصول المريض

PUT /api/appointments/{id}/status (Arrived/Waiting) يُضيف تلقائياً للطاير
يعمل لكن بدون معاملة!



6. استدعاء المريض

POST /api/clinic-queue/{id}/call يحدث الطاير ويزامن الموعد
يعمل



7. بدء الزيارة

POST /api/clinic-queue/{id}/start يُنشئ Visit تلقائياً ويزامن الموعد
يعمل



8. إكمال الزيارة

POST /api/clinic-queue/{id}/complete يكمل الطاير ويزامن الموعد
يعمل

6.2 تدفق الفاتورة والدفع

1. إنشاء فاتورة مسودة

Invoice (Status: Draft) يُنشئ PatientJourneyController.CreateDraftInvoice

لا يوجد POST /api/invoices لإنشاء فاتورة مستقلة



2. إصدار الفاتورة

Issued لـ PATCH /api/invoices/{id}/issue يغير الحالة

يعمل



3. تسجيل دفعة

POST /api/payments (مع InvoiceId) يُنشئ الدفعة ويربطها بالفاتورة ويحاول الانتقال لـ Paid تلقائياً

يعمل لكن الدفعة الواحدة لا يمكن توزيعها على عدة فواتير



4. استرداد دفعة

FinanceService.RefundPaymentAsync يُنشئ دفعة سلبية

لا يعيد حالة الفاتورة من Paid لـ Issued عند الاسترداد الكامل

6.3 نقاط الفشل الحرجة في الترابط

لا يوجد إنشاء تلقائي للمريض عند التأكد	Patient	BookingRequest
التحويل يتطلب Patient.Id يدوياً	Appointment	BookingRequest
إضافة الطابور بدون معاملة	ClinicQueue	Appointment
إلغاء الطابور لا يزامن الموعد	Appointment	ClinicQueue
تحديث دفعي يتجاوز إضافة الطابور	ClinicQueue	Appointment
علاقة واحد-متعدد فقط (لا كيان ربط)	Payment	Invoice
الاسترداد لا يعيد حالة الفاتورة	Invoice Status	Payment Refund
حقول مكررة (مصدران للبيانات)	ClinicQueue	Appointment

7. خطة الإصلاح حسب الأولوية

7.1 المرحلة الأولى: إصلاحات حرجية (فوري)

هذه الإصلاحات يجب تنفيذها فوراً لأنها تسبب فقدان بيانات أو سلوكاً خاطئاً ملموساً:

F1	إلغاء الطابور لا يزامن الموعد	إضافة SyncAppointmentStatus في cQueueController.Cancel
F2	تأكيد الحجز قد يؤكد بدون مريض (Frontend)	إذا فشل إنشاء المريض، لا تؤكد الحجز - أظهر خطأ
F3	لقطة النموذج تالفة	حذف الكتلة المكررة وإعادة توليد الترحيل التوحيقي
F4	تأكيد الحجز لا ينشئ مريضاً (Backend)	إضافة إنشاء تلقائي للمريض في ConvertToAppointment
F5	لا يوجد نقطة نهاية POST لإنشاء فاتورة	إضافة POST /api/invoices مع CreateInvoiceRequest

7.2 المرحلة الثانية: إصلاحات عالية الأولوية (خلال أسبوع)

H1	إضافة الطابور بدون معاملة	لف العمليتين في معاملة أو النقل للخدمة
H2	فحص تعارض الغرفة بعد حفظ الموعد	نقل الفحص قبل SaveChanges
H3	PaymentDto بدون CreatedAt/UpdatedAt	إضافة التعيين في MapPayment
H4	الاسترداد لا يعيد حالة الفاتورة	فحص إجمالي المدفوع بعد الاسترداد
H5	BookingRequest بدون PatientId FK	إضافة الحقل والتنقل والتكوين
H6	أنواع Frontend غير متطابقة	تحديث أنواع Dashboard, Queue, Finance, PaginatedResponse
H7	نموذج الموعد بدون ServiceId/ClinicRoomId	إضافة حقول اختيار الخدمة والغرفة
H8	حقول الحالة نصية سحرية	إنشاء تعدادات: ContractStatus, LabOrderStatus, SurgeryStatus
H9	InvoicePayment كيان ربط مفقود	إنشاء كيان InvoicePayment مع ترحيل

7.3 المرحلة الثالثة: إصلاحات متوسطة (خلال أسبوعين)

M1	استخراج طبقات خدمة لـ 8 متحكمات	.ClinicQueueService, InvoiceService, VisitService, etc
M2	تحسين استعلامات التقارير	GROUP BY على مستوى Include، DB، (لأطباء
M3	إنشاء ملفات تكوين لـ 40 كيان	أولوية: Contract, Receipt, PatientAccount, OrthoCase
M4	إضافة محقق DTOs	FluentValidation لجميع DTOs المطلوبة
M5	معالجة أخطاء Frontend	try-catch + toast لجميع استدعاءات API
M6	إزالة حقوق الطابور المكررة من Appointment	استخدام ClinicQueueItem كمصدر وحيد
M7	تحديث الشريط الجانبي	إضافة روابط لجميع الوحدات

7.4 ملخص الجهد التقديري

الأولى (حرجة)	5	2-3 أيام تطوير	فوري
الثانية (عالية)	9	5-7 أيام تطوير	خلال أسبوعين
الثالثة (متوسطة)	7	7-10 أيام تطوير	خلال أسبوعين
الإجمالي	21 إصلاح أساسي	14-20 يوم تطوير	3 أسابيع

هذه الخطة تركز على 21 إصلاحاً أساسياً من أصل 67 مشكلة مكتشفة. باقي المشاكل (46) ذات أولوية منخفضة أو ويمكن معالجتها بالتدرج في المراحل اللاحقة. الأولوية القصوى لإصلاح المشاكل الحرجة التي تسبب فقدان بيانات ملموساً للمستخدمين، خاصة مشكلة إلغاء الطابور بدون مزامنة الموعد ومشكلة تأكيد الحجز بدون مريض.